ZECPATH AI SYSTEM

Internship Technical Report

DAY 6

Job Description Reading, Parsing & Pydantic Validation

Submitted By	Sanjay Manoj K
Organization	Zecser Technology
Project	AI-Based Applicant Tracking System
Status	**Completed**
Focus Area	ATS Input Processing — JobRequirement Object
Source File	data/job_descriptions/python_developer.txt
Technology	Python • Text Parsing • Pydantic

SCOPE: Day 6 only — Job Description reading, structured extraction and Pydantic validation. Prepared as an internship submission document.

Table of Contents

Screenshots and diagrams in this report are reconstructed from terminal output and design specifications supplied during the Day 6 implementation. No unprovided UI state has been invented.

1. Executive Summary

Day 6 focused exclusively on processing a Job Description (JD) and converting it from raw text into a structured, validated representation. The implemented pipeline reads the Python Developer job description, parses its role, required skills, experience, education and responsibilities, and validates the resulting dictionary using a Pydantic **JobRequirement** model.

Capability	Implementation	Result
JD reading	read_job_description()	Raw JD text successfully loaded
JD parsing	parse_job_description()	Structured fields extracted
Data validation	JobRequirement(**job_requirements)	Validation successful
Structured output	Role, skills, experience, education, responsibilities	Ready for ATS matching

2. Objectives & Scope

Objective	What was done	Status
Read JD from file	Loaded data/job_descriptions/python_developer.txt	✓ Completed
Identify requirements	Separated role, skills, experience, education and responsibilities	✓ Completed
Create structured data	Produced job_requirements dictionary	✓ Completed
Validate structure	Created JobRequirement Pydantic object	✓ Completed
Prepare downstream input	Exposed validated fields for future ATS matching	✓ Ready

3. Implementation

3.1 Job Description Input

The tested input is a plain-text job description for a Python Developer. It contains a role title, five technical requirements, an experience requirement, an education requirement and five responsibilities.

```
===== JOB DESCRIPTION =====
Python Developer
We are looking for a Python Developer to join our development team.
Requirements:
• 2+ years of experience in Python development.
• Strong knowledge of Python and Django.
• Experience with REST APIs and SQL.
• Good understanding of Git and version control.
• Bachelor's degree in Computer Science or a related field.
Responsibilities:
• Develop and maintain Python applications.
• Build and integrate REST APIs.
• Work with databases using SQL.
• Collaborate with the development team.
• Write clean and maintainable code.
```

Figure 1. Day 6 terminal execution showing the Job Description read by the system.

3.2 Structured Requirement Extraction

Field	Extracted value
Role	Python Developer
Required skills	Python; Django; REST APIs; SQL; Git
Experience	2+ years of experience
Education	Bachelor's degree in Computer Science or a related field
Responsibilities	Develop and maintain Python applications; build and integrate REST APIs; work with databases using SQL; collaborate with the development team; write clean and maintainable code.

3.3 Pydantic Validation

The extracted dictionary is passed to the **JobRequirement** Pydantic model. Validation confirms types, required fields and data integrity. The console reports “Job Requirement validation successful!” and prints the structured object ready for the matching engine.

4. Architecture & Workflow

Day 6 adds the Job Description processing branch of the ATS system. Its output is a validated JobRequirement object that can later be consumed by the matching and scoring engine.

4.1 Day 6 Architecture

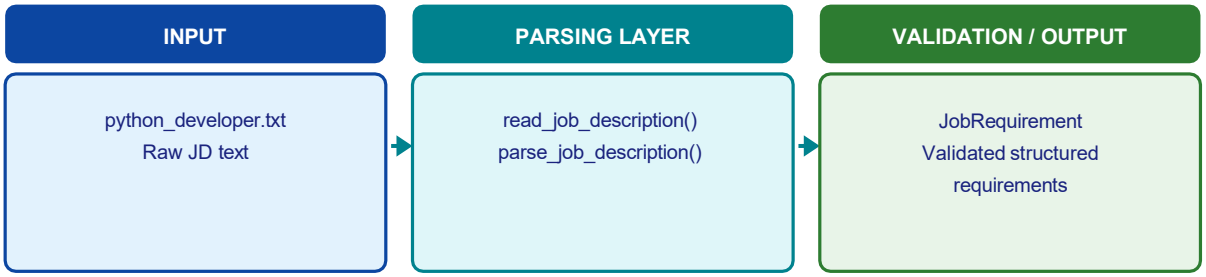


Figure 2. High-level architecture of the JD processing pipeline.

4.2 Workflow Steps

Step	Action	Output
1	Read job description file	Raw JD text
2	Parse JD sections	Structured dictionary
3	Instantiate JobRequirement	Pydantic model
4	Validate and display	Verified requirements

4.3 Downstream Connection

The validated JobRequirement is the contract between JD processing and the future ATS matching engine. The next layer can compare **required_skills**, **experience** and **education** against the validated CandidateProfile produced on Day 5.

5. Process Diagrams

The following diagrams illustrate the complete data flow from raw job-description text through parsing and Pydantic validation to a ready-to-use JobRequirement object.

5.1 End-to-End Pipeline

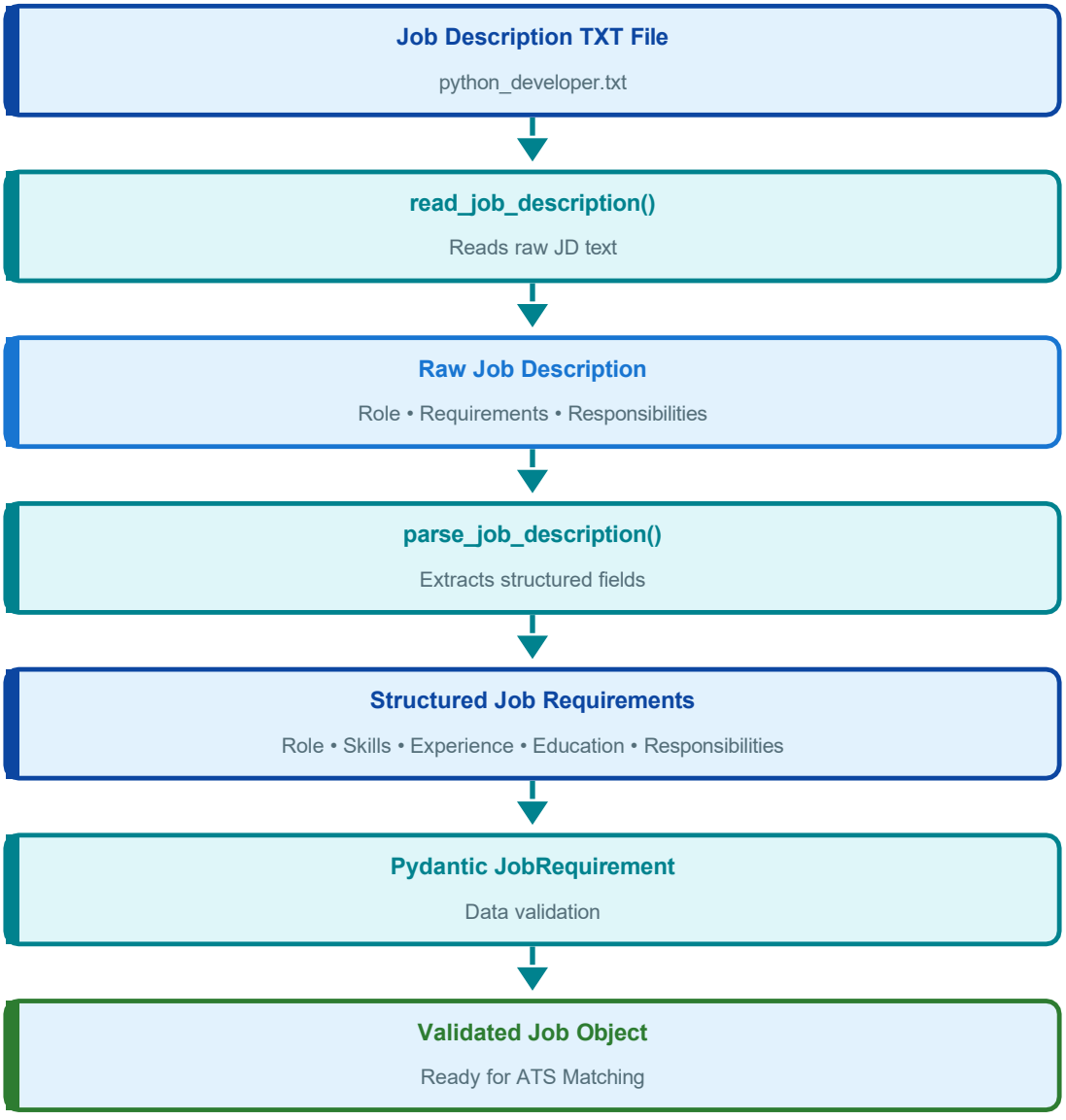


Figure 3. Vertical pipeline from file input to validated JobRequirement object.

5.2 Raw → Structured → Validated Flow

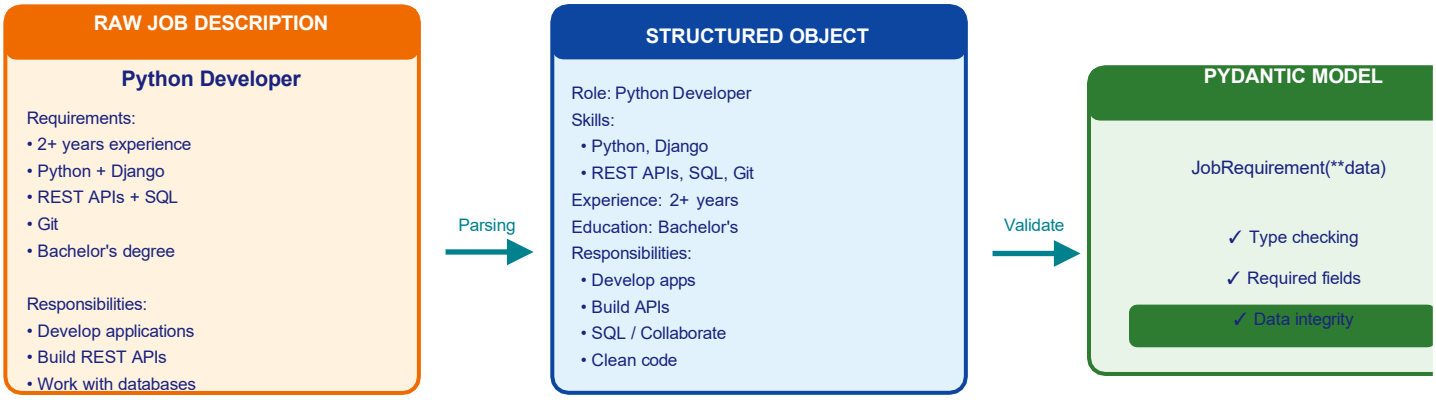
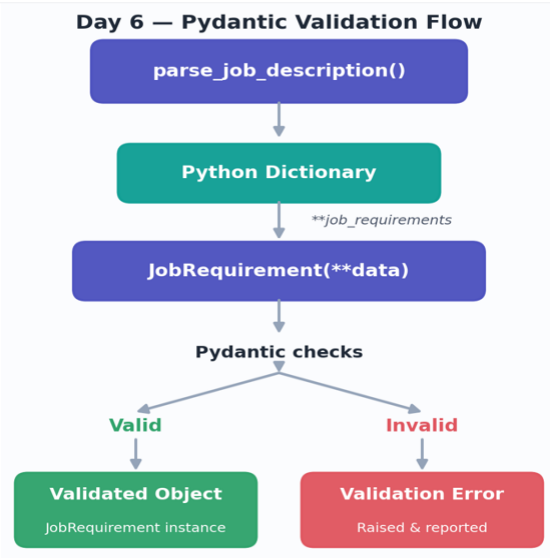


Figure 4. Transformation of raw JD text into a validated structured object.



Pydantic raises a validation error on the first invalid or missing field; otherwise it returns a typed JobRequirement instance.

5.3 JobRequirement Object Tree

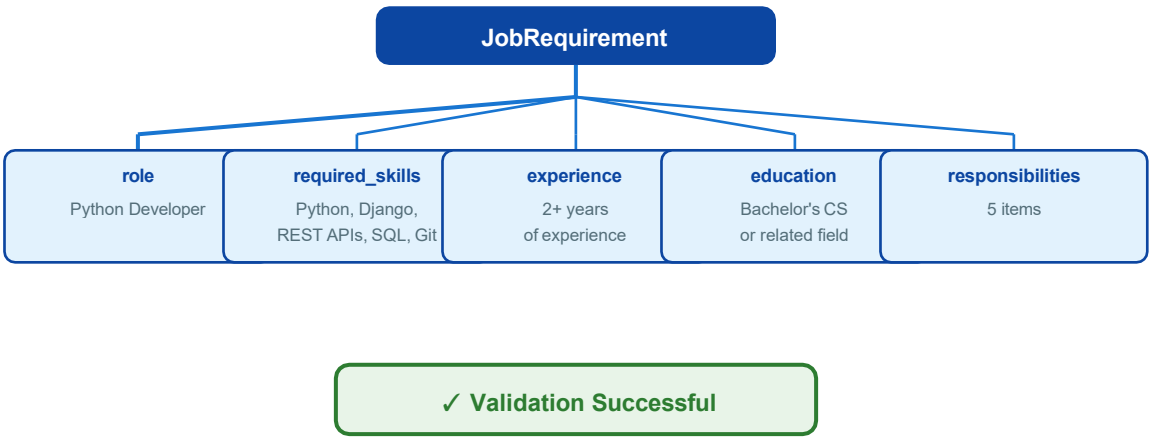
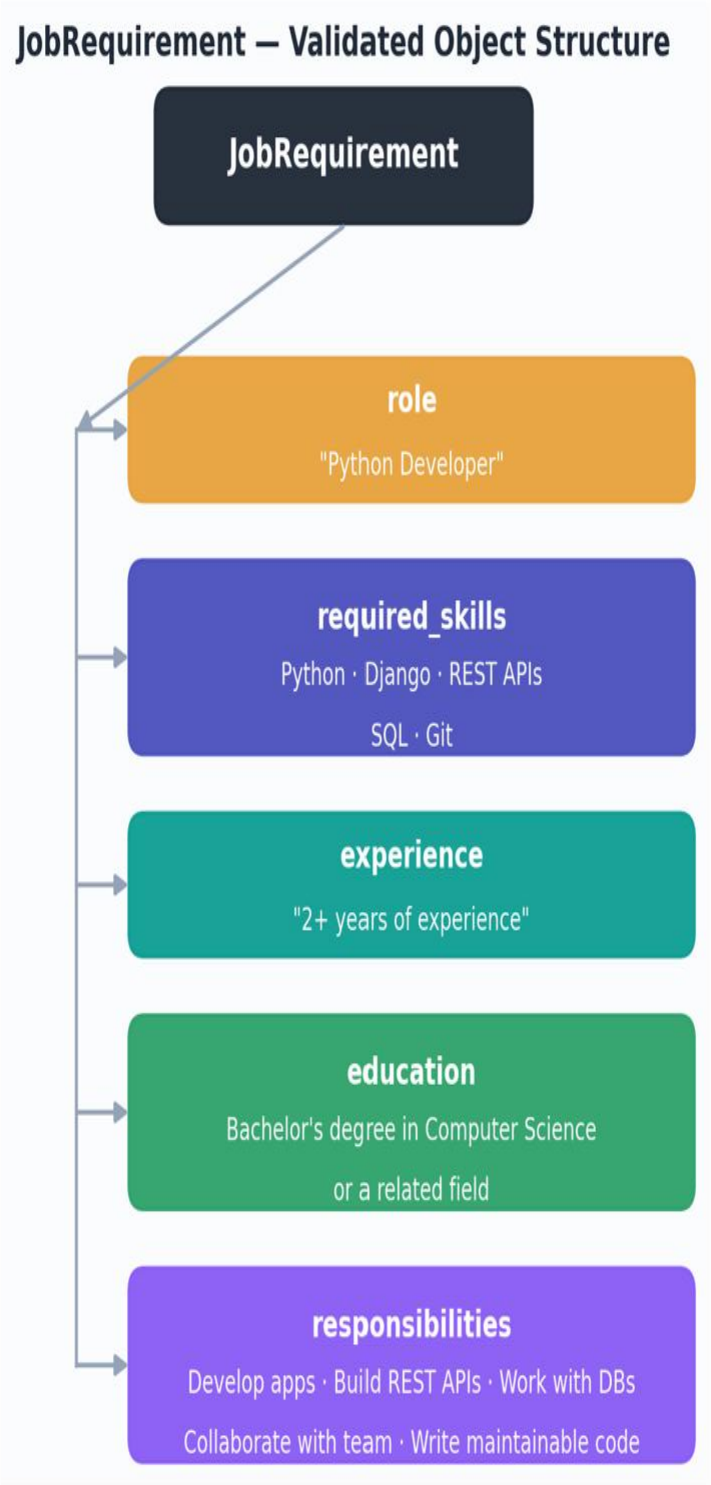


Figure 5. Structure of the validated Pydantic JobRequirement model.

Validated Object Structure

The final JobRequirement instance exposes five typed fields, ready to be consumed by the downstream matching engine.



Field-level structure of the validated JobRequirement object.

6. Code Snippets

6.1 Imports

```
from parsers.jd_parser import read_job_description, parse_job_description
from ats_engine.ats_engine.job_requirement import JobRequirement
```

6.2 Read and Parse the Job Description

```
jd_file_path = "data/job_descriptions/python_developer.txt"

jd_text = read_job_description(jd_file_path)

job_requirements = parse_job_description(jd_text)
```

6.3 Pydantic Validation

```
job_requirement = JobRequirement(**job_requirements)

print("\n===== PYDANTIC JOB VALIDATION =====")
print("Job Requirement validation successful!")

print("Role:", job_requirement.role)
print("Required Skills:", job_requirement.required_skills)
print("Experience:", job_requirement.experience)
print("Education:", job_requirement.education)
print("Responsibilities:", job_requirement.responsibilities)
```

6.4 Structured Output Display

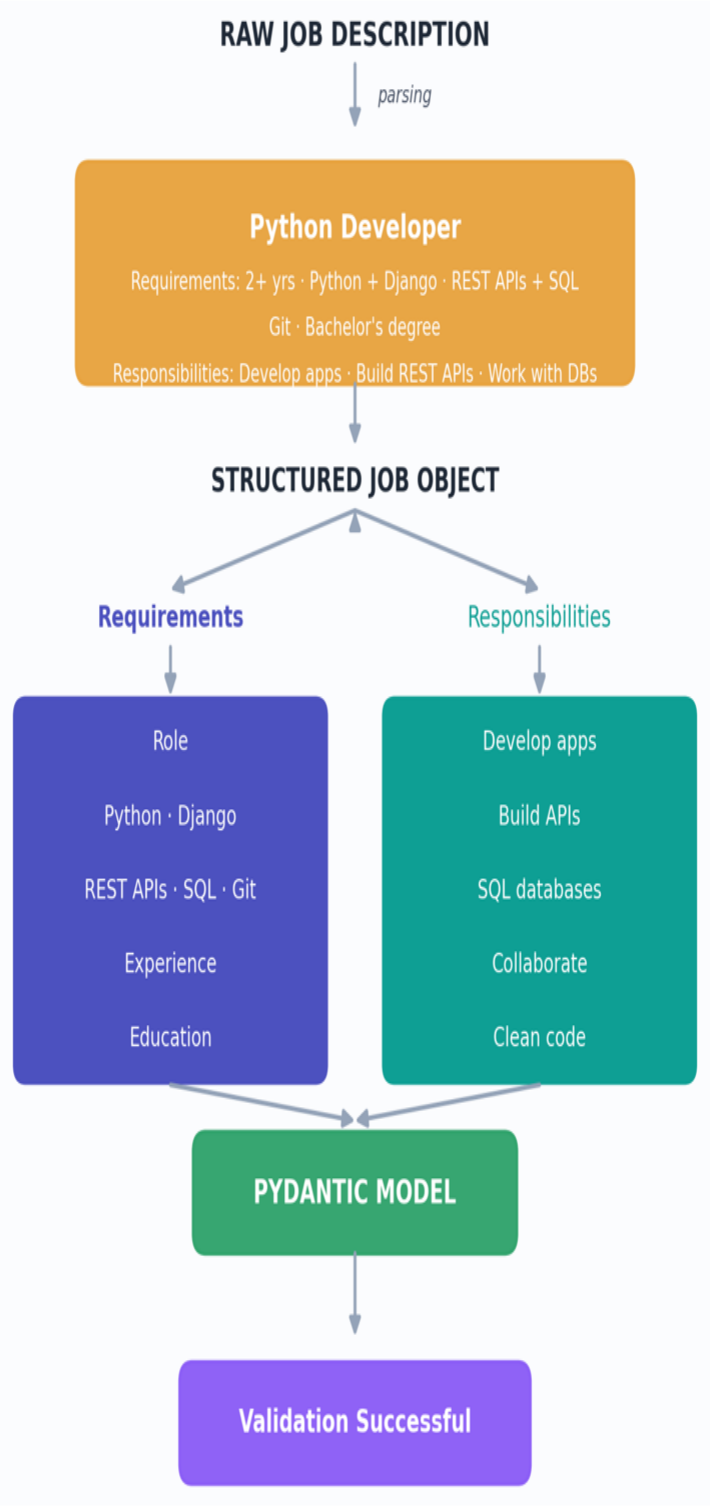
```
print("\n===== STRUCTURED JOB REQUIREMENTS =====")
print("Role:", job_requirements["role"])

print("Required Skills:")
for skill in job_requirements["required_skills"]:
print("-", skill)

print("Experience:", job_requirements["experience"])
print("Education:", job_requirements["education"])

print("Responsibilities:")
for responsibility in job_requirements["responsibilities"]:
print("-", responsibility)
```

The implementation follows a clear separation of concerns: file reading, parsing, validation and presentation are

distinct stages.

Requirement Decomposition

Once read, the raw description is split into a Requirements branch and a Responsibilities branch before both are merged back into a single validated object.



7. Challenges & Solutions

Challenge	Observed during implementation	Solution	Outcome
Incorrect import path	ModuleNotFoundError while locating JobRequirement	Corrected import to nested ats_engine package path	Execution restored
Duplicate / misplaced execution code	JD processing code temporarily outside main()	Moved complete Day 6 workflow into main()	Clean execution
Pydantic output visibility	Validation existed but was not obvious in console	Added explicit validation-success and structured-object sections	Verification is clear
Raw JD vs structured data	Raw text alone is difficult for downstream ATS logic	Introduced parse_job_description() and JobRequirement	Reusable structured contract

8. Deliverables & Verification

Deliverable	Verification evidence	Status
JD reader	python_developer.txt successfully printed under JOB DESCRIPTION	✓ Complete
JD parser	Role, skills, experience, education and responsibilities displayed	✓ Complete
JobRequirement model	Pydantic object printed with all required fields	✓ Complete
Pydantic validation	"Job Requirement validation successful!"	✓ Complete
Structured requirements	Readable section-by-section output	✓ Complete

DAY 6 DELIVERABLES READY

Job Description reading, parsing, structured requirement extraction and Pydantic validation are implemented and verified through successful terminal execution. The JobRequirement object is ready to be consumed by the ATS matching engine.

9. Next Steps

Priority	Next development task	Purpose
1	Connect JobRequirement with CandidateProfile	Enable candidate-to-JD comparison
2	Implement skill matching	Identify matched and missing skills
3	Implement experience matching	Compare required vs candidate experience
4	Implement education matching	Evaluate educational eligibility
5	Create weighted ATS score	Produce an interpretable overall score
6	Add gap analysis and recommendations	Explain why a candidate matches or misses requirements
7	Test with multiple JD formats	Improve parser robustness

Conclusion

Day 6 successfully establishes the Job Description side of the structured ATS pipeline. The system can read the JD, extract its key requirements, validate them using Pydantic and expose a clean JobRequirement object. This creates the required foundation for the next stage: automated resume-to-job matching and ATS scoring.

END OF DAY 6 REPORT

Zecpath AI System • Zecser Technology • Sanjay Manoj K